GPU-Accelerated Graph-Colored Simulated Annealing for Integer Factorization

Advith Desu, Aryan Namboodri, and Anil Prabhakar
Centre for Quantum Information, Communication and Computing, Indian Institute of Technology Madras, Chennai 600036, India
(Dated: June 9, 2026)

The security of RSA rests on the hardness of factoring a semiprime $N = PQ$. We present an end-to-end pipeline that maps factorization to a sparse Ising model, solve it on a single NVIDIA GH200 GPU with a graph-colored simulated annealer, and close the residual gap with a guided brute-force search. Together we are able to reliably able to factor ~ 100-bit semiprimes.

I. INTRODUCTION

Multiplying two large primes is easy; recovering them from their product $N = PQ$ is hard. This one-way asymmetry is the foundation of RSA, the most widely deployed public-key cryptosystem [1]. Shor's algorithm [2] would undo it on a fault-tolerant quantum computer, but the qubit counts required at cryptographic key sizes remain extremely high [3]. This work explores a classical approach to factoring (framed as an optimization problem), that is worth investigating.

Factoring $N = PQ$ can be cast as an optimization problem; an Ising model whose lowest-energy spin configuration encodes the bits of P and Q [4, 5]. Recovering the factors then reduces to finding that ground state. This is the task of an Ising machine, a solver built to minimise an Ising energy, realised today in classical [6], quantum [7], and quantum-inspired [8] hardware. We use simulated annealing [9, 10] for this task.

This work mainly focuses on the optimal implementation of the annealer. Simulated annealing repeats the same simple update across every spin, many times over, a workload that maps naturally onto a GPU, where thousands of these updates can run in parallel. We shape the implementation around the structure of the problem at every level, from how the model is laid out in memory to how the work is assigned to the GPU's threads, so that as little of the hardware as possible sits idle. The sections that follow develop each of these choices.

II. INTEGER FACTORIZATION AS A BINARY OPTIMIZATION PROBLEM

Let $N = PQ$ be a semiprime with P, Q being primes of equal bit length $n = \lceil \frac{1}{2} \log_2 N \rceil$. Write,

$$P = \sum_{k=0}^{n-1} 2^k p_k, \qquad Q = \sum_{k=0}^{n-1} 2^k q_k, \qquad p_k, q_k \in \{0, 1\}. \tag{1}$$

Four bits are known a priori: $p_0 = q_0 = 1$ (as both factors are odd) and $p_{n-1} = q_{n-1} = 1$ (each is exactly n bits).

Multiplying P and Q in their binary representations and matching the result against the known bits of N turns factorization into a constraint satisfaction problem. We adopt the column-based multiplication-table formulation, and the classical bit-reduction rules that accompany it, from Jiang *et al.* [4], who used the same construction to factor semiprimes on a quantum annealer. Section III develops the column equations, the rules that follow them, and the conversion to an Ising model that encodes P and Q.

III. CLASSICAL PRE-PROCESSING

Multiplying P and Q is done using a multiplication table, Table I lays it out for $N = 391 = 17 \times 23$. The cross-terms $p_a q_b$ land in the column of place value 2^{a+b}. The contribution of a given place value to the product is the sum of every entry in its column.

Since that column sum can exceed one, there would be a carry into higher columns. We make these carries explicit as binary variables: $s_{i,j} \in \{0, 1\}$ is the bit of column i's carry that arrives in column $j > i$, where it contributes with weight 2^{j-i}. Every column thus receives carries from lower columns (its carry-ins) and sends carries to higher ones (its carry-outs).

Reading a column means balancing everything that lands in that place value against the known bit N_i of N. Collecting the partial products, the incoming carries, and the outgoing carries gives one equation per column:

$$C_i = N_i - \sum_j q_j \, p_{i-j} - \sum_j s_{j,i} + \sum_j 2^j \, s_{i,i+j} = 0, \tag{2}$$

The true factors are the unique bit assignment for which every column is balanced, i.e. $C_i = 0$ for all i. Squaring each clause and summing it gives us the energy function:

$$E(\{p_i, q_i, s_{ij}\}) = \sum_i C_i^2, \tag{3}$$

whose global minimum $E = 0$ encodes the true factors P and Q.

Squaring (2) directly, however, produces a polynomial with a large number of terms. Before handing the problem to the annealer, we try to eliminate as many variables and as much redundancy as possible using deterministic algebraic rules [4].

TABLE I. Binary long-multiplication table for $N = 391 = 17 \times 23$, with $P = (1\,p_3\,p_2\,p_1\,1)_2$ and $Q = (1\,q_3\,q_2\,q_1\,1)_2$ (the outer bits are fixed; only $p_1, p_2, p_3, q_1, q_2, q_3$ are unknown). Each cross-term $p_a q_b$ sits in the column of place value 2^{a+b}. A column also receives carries from lower columns (carry-in) and emits carries to higher ones (carry-out). Balancing the column sum against the known bit of N gives the clause $C_i = 0$ of Eq. (2).

	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
P					1	p_3	p_2	p_1	1
Q					1	q_3	q_2	q_1	1
					1	p_3	p_2	p_1	1
				q_1	p_3q_1	p_2q_1	p_1q_1	q_1	
			q_2	p_3q_2	p_2q_2	p_1q_2	q_2		
		q_3	p_3q_3	p_2q_3	p_1q_3	q_3			
	1	p_3	p_2	p_1	1				
carry-in	$s_{6,8}$	$s_{5,7}$	$s_{4,6}$	$s_{3,5}$	$s_{2,4}$	$s_{2,3}$	$s_{1,2}$		
	$s_{7,8}$	$s_{6,7}$	$s_{5,6}$	$s_{4,5}$	$s_{3,4}$				
carry-out		$s_{7,8}$	$s_{6,7}$	$s_{5,6}$	$s_{4,5}$	$s_{3,4}$	$s_{2,3}$	$s_{1,2}$	
			$s_{6,8}$	$s_{5,7}$	$s_{4,6}$	$s_{3,5}$	$s_{2,4}$		
$N = 391$	1	1	0	0	0	0	1	1	1

A. Reduction rules

A small set of deterministic rules is enough to drive most carries and several factor bits to fixed values, or to rewrite one bit as an expression in others. We use seven such rules: the saturated-sum, zero-sum, doubling, coefficient-bound, complement, parity and replacement rules. The simplifier applies them in a fixed priority order, looping until no rule fires on any column clause. They are detailed in Appendix A.

B. Quadrization

The energy (3) is degree four in the binary variables. Cubic and quartic terms arise from squaring clauses that contain products like $p_i q_j$. Standard QUBO solvers (and Ising hardware) require degree two, so we apply Rosenberg substitutions [11]: introduce an auxiliary w and add a penalty $P(A, B, w) = M(AB - 2Aw - 2Bw + 3w)$ with M large enough that the minimizer would set $w = AB$. After substitution every cubic $AB \cdot s$ becomes a quadratic $w \cdot s$ plus a constant-size quadratic penalty. The two sign variants of the cubic case, and the two variants of the quartic case, are standard and whose implementation follows [4].

C. From QUBO to Ising

A linear change of variables $\sigma_i = 1 - 2x_i$ maps each binary $x_i \in \{0, 1\}$ to an Ising spin $\sigma_i \in \{-1, +1\}$. We store Q in symmetric form ($Q_{ij} = Q_{ji}$), with the energy function then being represented as,

$$H = \sum_i Q_{ii} x_i + \sum_{i \neq j} Q_{ij} x_i x_j. \qquad (4)$$

Substituting $x_i = (1 - \sigma_i)/2$ gives

$$H(\boldsymbol{\sigma}) = \sum_i h_i \sigma_i + \sum_{i \neq j} J_{ij} \sigma_i \sigma_j + \text{const.}, \qquad (5)$$

with the coupling and field taken over the full (symmetric) sum,

$$J_{ij} = \tfrac{1}{4} Q_{ij}, \quad h_i = -\tfrac{1}{2} \sum_j Q_{ij}. \qquad (6)$$

The Ising form of the factorization problem is what we use throughout.

IV. STRUCTURAL SCALING AND SPARSE STORAGE

a. The coupling matrix is sparse, and stays sparse. Figure 1 plots the nonzero structure of J_{ij} for three representative semiprimes. The dense block corresponds to auxiliary w-variables introduced during quadrization (Section III B), each of which couples to every clause it participates in and the diagonal trail comes from the carries $s_{i,j}$.

b. CSR storage. We store J in Compressed Sparse Row (CSR) format [12]: a `row_ptr` of $N + 1$ row offsets, a `col_idx` array of column indices, and a `values` array of the nonzeros. Memory is $\mathcal{O}(N + \text{nnz})$ rather than $\mathcal{O}(N^2)$. Here, nnz is the number of nonzeros values in the matrix.

V. GRAPH-COLORED SIMULATED ANNEALING ON THE GPU

The annealer is a Metropolis sampler that tries to flip every spin once per sweep, accepting flips with probability $\min(1, e^{-\beta \Delta E_i})$. To fully make use of the parallelism

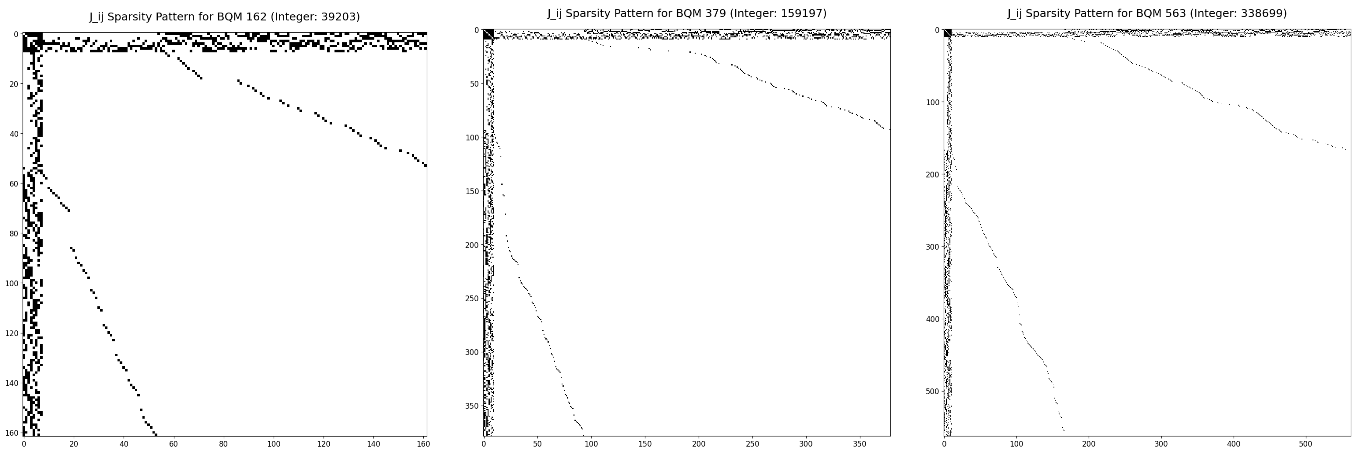

FIG. 1. Sparsity of J_{ij} at $N = 39\,203$ (162 spins), $159\,197$ (379), and $338\,699$ (563). Most nonzeros concentrate in a small band along the top rows / left columns, with a thin diagonal tail. Pattern persists across orders of magnitude in N.

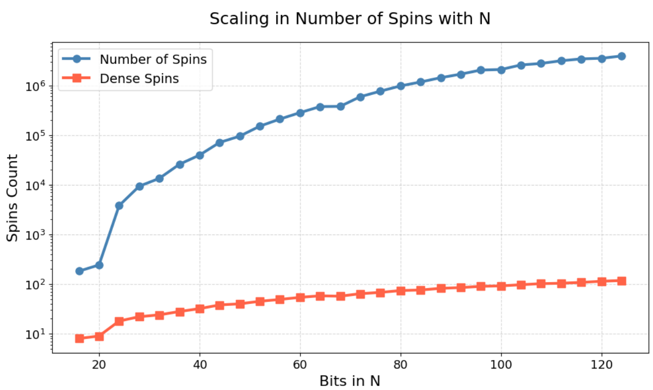

FIG. 2. Number of total Ising spins versus number of densely connected hub spins as the bit width of N grows to ~ 128.

of a GPU we need three things: a way to flip many spins in parallel without breaking the energy bookkeeping, kernels whose work pattern matches GPU resources, and a memory layout that keeps the highly accessed data in fast storage.

A. Computing ΔE from local fields

Each Metropolis test needs the energy change ΔE_i that flipping spin i would cause. The straightforward way to calculate it is to evaluate the total energy of the current state, flip the spin, evaluate the total energy again, and subtract. This however, is not effecient.

Flipping a single spin, changes only the energy terms that involve that spin. Writing the Ising energy as $E(\boldsymbol{\sigma}) = \sum_{i<j} J_{ij}\sigma_i\sigma_j + \sum_i h_i\sigma_i$ and flipping $\sigma_i \to -\sigma_i$ changes it by

$$\Delta E_i \;=\; -2\,\sigma_i \underbrace{\Big(\textstyle\sum_j J_{ij}\sigma_j + h_i\Big)}_{\text{local field } L_i}, \tag{7}$$

where L_i is the local field acting on spin i. The ΔE_i calculation therefore collapses to a single sparse dot product over spin i's neighbour row.

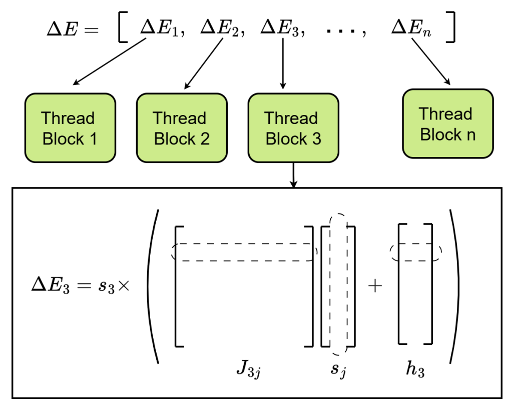

FIG. 3. Local-field evaluation of ΔE_i. Each thread group loads spin i's neighbour row from the CSR, multiplies each coupling J_{ij} by the neighbour spin σ_j, reduces the products to the local field $L_i = \sum_j J_{ij}\sigma_j + h_i$, and returns $\Delta E_i = -2\sigma_i L_i$. One group per spin lets all N differences be evaluated in a single parallel pass.

This local-field form is what makes the GPU useful. Since L_i depends only on the current configuration and never on another candidate's flip, the ΔE_i of all N spins are mutually independent and can be evaluated in a single parallel pass, with one thread group reducing each spin's neighbour row (Figure 3. The thread-to-spin mapping is the subject of Section V C). A full sweep then costs only $\mathcal{O}(\mathrm{nnz})$ of total work, spread across the GPU.

Flipping every accepted spin at once would cause errors in the final energy value. All the ΔE_i are computed against the same frozen configuration. If two coupled spins ($J_{ij} \neq 0$) are both accepted and flipped together, each one's ΔE assumed the other stayed fixed, and the running energy accumulator no longer matches the new state. We may only flip a set of spins that are mutually uncoupled, and finding such sets is exactly what graph coloring provides.

B. Why parallel Metropolis needs graphcoloring

Coloring of the interaction graph (nodes = spins, edges = nonzero J_{ij}) produces the mutually-uncoupled sets we look for. It partitions the spins into independent sets (colors) such that no two same-color spins are coupled ($J_{ij} = 0$ for all i, j of the same color). All spins of one color can then be flipped in parallel: each one's ΔE depends only on spins of other colors, which are unchanged during this color's pass. Sweeping every color in sequence gives us a full Metropolis sweep.

We compute the coloring once at setup using the Welsh–Powell greedy heuristic [13].

C. Further optimizations in implementation

Two further choices keep the parallel sweep fast in practice. We introduce them here, with full details in Appendix B.

First, each spin is handled by a dedicated group of GPU threads, sized to its work. Computing a spin's ΔE is a reduction over its neighbour row (Eq. (7)), so the natural amount of parallelism is the length of that row (non-zero values in the row). A densely connected spin is therefore assigned a full thread block (1024 threads). Every other spin is handled by a single warp (32 threads). Matching the group size to the row length avoids both idle threads on short rows and serialized work on long ones.

Second, the densely connected spins are cached in fast on-chip memory. Since, they couple to many others, they are read by almost every other spin's local-field computation, on every pass. Fetching them from slow global memory each time would then be ineffecient. At the start of a pass, we copy their current values into the shared memory local to each thread group, where every group can read them cheaply.

Algorithm 1: One-time setup before annealing

- Verify J symmetry and zero diagonal;
- Bin each spin: **dense** if $d_i \geq 128$, else **sparse**;
- color$[\cdot] \leftarrow$ Welsh–Powell greedy coloring;
- Bucket spins by color into dense / sparse offset arrays;
- Build the sparse-spin CSR and the dense-spin cache;
- Upload CSR, color tables, and dense-spin arrays to the GPU;
- Initialize a random spin state and compute the initial energy;
- Estimate $T_0 = \langle \Delta E^+ \rangle / \ln(1/\chi)$, where χ is acceptance rate;

Algorithm 2: Color-parallel simulated annealing loop

foreach *β in the geometric schedule* **do**
  for sweep $= 1 \ldots m$ **do**
    Draw a uniform random $r_i \in [0, 1)$ for every spin;
    for $c = 0 \ldots N_{\text{colors}} - 1$ **do**
      `/* score every spin of color` c `in parallel */`
      foreach *spin i of color c* (in parallel) **do**
        $L_i \leftarrow \sum_j J_{ij}\, \sigma_j + h_i$;
        $\Delta E_i \leftarrow -2\, \sigma_i L_i$;
        `/* Metropolis acceptance test */` **if** $r_i < \min\big(1,\, e^{-\beta \Delta E_i}\big)$ **then**
          mark spin i as accepted;
      `/* apply all accepted flips of the color together */`
      foreach *accepted spin i* (in parallel) **do**
        $\sigma_i \leftarrow -\sigma_i$;
        add ΔE_i to the running energy;
      if *color c contained a dense spin* **then**
        refresh the dense-spin cache;
  if *running energy $<$ best energy* **then**
    save the current spins as the best;
return best-energy spin configuration;

VI. BRUTE-FORCE POST-PROCESSING

The annealer rarely lands exactly on the ground state at the size of bits we test. It does, however, land close: post-processing of the final spin output recovers approximate factors $(P_{\text{pred}}, Q_{\text{pred}})$ that differ from the true (P, Q) by a few percent (Section VII). A targeted brute-force search closes the residual gap.

a. Centering. The annealer provides two independent estimates of P: P_{pred} from the spins directly, and $\lfloor N/Q_{\text{pred}} \rfloor$ from the complementary factor. Their average is the search centroid:

$$P_{\text{centre}} \;=\; \tfrac{1}{2}\big(P_{\text{pred}} + \lfloor N/Q_{\text{pred}} \rfloor\big). \tag{8}$$

Empirically, this gives us a better starting point (see Figure 6).

b. Outward-spiral search. The search runs on a CPU, spread across all available threads. Candidates around P_{centre} are grouped into fixed-size chunks of 2^{14} consecutive integers. Each chunk is indexed by $k = 0, 1, 2, \ldots,$ and visited in outward-spiral order so that the candidates closest to P_{centre} are tested first. Each thread repeatedly claims the next chunk through a shared atomic counter.

c. Wheel sieve. Checking whether a candidate p divides N is an expensive operation, so we want to do it as rarely as possible. The factor we are looking for is a large prime, and a prime is never a multiple of a smaller prime, so any candidate p divisible by 2, 3, 5, 7, or 11 can be thrown out at once, with no division by N. The integers that survive this filter repeats with period $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 = 2310$, and only 480 of every 2310 ($\approx 21\%$) survive. We precompute those 480 surviving integers once; each chunk then performs the checks only on the survivors and skips the remaining.

d. Early termination. The first thread to find $p \mid N$ sets a shared stop flag; all other threads exit at the next chunk boundary. The expected wall-clock cost follows the empirical model

$$T_{\text{bf}} \approx \frac{3.32 \times 10^{-9}}{\text{threads}} \cdot \text{Gap}(\%) \cdot 2^{\lceil n/2 \rceil}, \qquad (9)$$

where n is the bit length of the semi-prime (N) and Gap(%) is the relative distance between P_{centre} and the true P. The exponential in n is unavoidable; the only parameter the annealer controls is Gap(%), indicative of solution quality.

VII. RESULTS

All experiments run on a single NVIDIA GH200 Grace Hopper Superchip. The annealing kernel is compiled with `nvcc` (CUDA 12.x); QUBO construction and brute-force post-processing run on the Grace CPU side. Cooling schedule: geometric with $\alpha = 0.95$, T_0 auto-estimated at $\chi = 0.6$, $T_{\min} = 10^{-3}$, 10 sweeps per β step.

For each bit width $n \in \{16, 20, 24, \ldots, 124, 128\}$ we generate a random n-bit semiprime $N = PQ$ with P, Q random $\lceil n/2 \rceil$-bit primes, build the QUBO once, and run the SA ten times with independent seeds. Reported numbers use the winning seed.

A. Per-iteration throughput

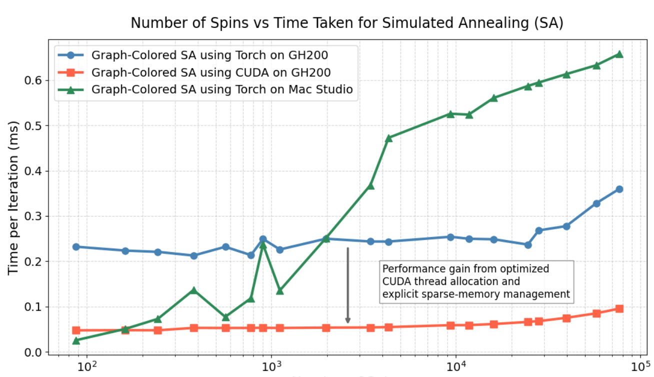

FIG. 4. SA wall-clock per sweep iteration versus number of Ising spins, comparing the CUDA kernel on GH200, a PyTorch implementation on GH200, and the same PyTorch code on an Apple M2 Max via MPS.

Figure 4 compares wall-clock per sweep across the three implementations. The CUDA kernel stays near 0.05 ms per iteration across the spin count; the PyTorch implementation of graph colored simulated annealing, pays roughly 5× that on the same GPU because its primitives do not exploit the structure of the problem as we do in Section V C.

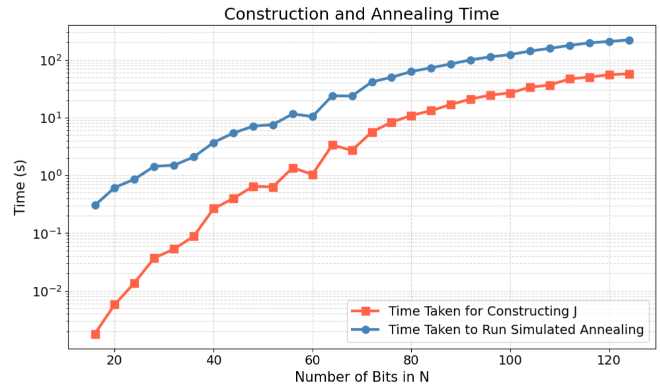

FIG. 5. Time to build J and time to run one full annealing schedule, both as a function of n in $[16, 128]$ bits.

B. Solution quality

Figure 6 reports the relative gap between the predicted and the true factor for three estimators:

1. A naive estimate $\sqrt{N}$. Mean gap 10.33% over the tested range.

2. P_{pred} from the SA spin state directly. Mean gap 4.23%.

3. The averaged centroid $\frac{1}{2}(P_{\text{pred}} + \lfloor N/Q_{\text{pred}} \rfloor)$. Mean gap 2.98%.

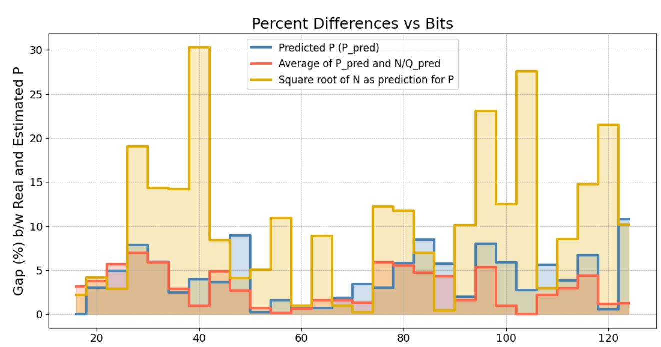

FIG. 6. Percentage gap between predicted and true P as a function of n, for three estimators: $\sqrt{N}$ (baseline), P_{pred} from SA, and the averaged centroid $\frac{1}{2}(P_{\text{pred}} + N/Q_{\text{pred}})$.

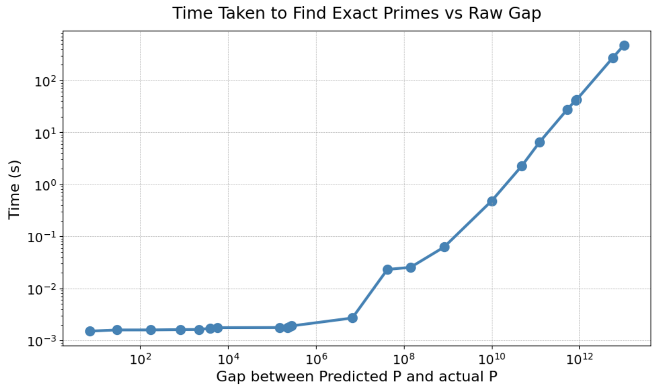

FIG. 7. Time taken to find the exact factor by the wheel-sieve brute-force search, as a function of the raw gap between P_{centre} and the true P, on the 72-thread GH200 Grace CPU.

C. Brute-force time as a function of gap

Figure 7 shows two regimes, both a consequence of the chunk-per-thread parallelism of Section VI. The search hands out fixed-size chunks to the 72 threads of the GH200 Grace CPU. When the gap is small, the entire search space is only a handful of chunks: every thread takes at most one, and a single parallel pass already brackets the true factor. The time is then just the fixed cost of that one pass, independent of the exact gap (the flat region below a gap of $\sim 10^5$). Once the gap grows past what 72 threads can cover at once, the threads run out and must work through the surplus chunks in successive rounds; the search effectively becomes serial, and the time grows linearly with the gap, matching the $\mathrm{Gap} \cdot 2^n$ term of Eq. (9).

D. ~ 100-bit end-to-end wall-clock time

For

$$N = 416\,749\,328\,744\,899\,275\,664\,046\,210\,629,$$

the SA-guided pipeline reports:

Stage	Wall-clock
Build J matrix (CSR)	25.3 s
Simulated annealing ($\alpha = 0.95$, auto T_0)	122.8 s
Brute-force from $P_{\mathrm{centre}} = 5.79 \times 10^{14}$	267.9 s
Total	**416.0 s**

The naive $\sqrt{N}$-seeded baseline on the same 72-thread CPU takes 3323.8 s. The recovered factors are $P = 573\,858\,364\,692\,161$ and $Q = 726\,223\,323\,360\,389$.

VIII. DISCUSSION

A. Limitations

a. Solution-quality gap grows with n. The mean gap of 4.23% (raw SA) is across 16–128 bits; the individual-instance gap is noisier at the upper end and we observed runs with $> 8\%$ gap at 120+ bits. The ~ 100-bit case study is representative, not best-case.

b. Brute-force scalability. At cryptographic key sizes (1024+ bits), a $\mathrm{Gap} \cdot 2^n$ brute-force cost quickly becomes unfeasible. The right question, which we leave to future work, is what factor-finding algorithm best exploits a P_{centre} within a few percent of the truth. Pollard's ρ and Lenstra's elliptic curve method are both candidates that benefit much more from a good starting point than linear scan does.

[1] R. L. Rivest, A. Shamir, and L. Adleman, A method for obtaining digital signatures and public-key cryptosystems, Communications of the ACM **21**, 120 (1978).

[2] P. W. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, SIAM Journal on Computing **26**, 1484 (1997).

[3] N. Jain *et al.*, Information-theoretic and computationally secure cryptography in the quantum era, Contemporary Physics **57**, 366 (2016).

[4] S. Jiang, K. A. Britt, A. J. McCaskey, T. S. Humble, and S. Kais, Quantum annealing for prime factorization, Scientific Reports **8**, 10.1038/s41598-018-36058-z (2018).

[5] B. Wang, F. Hu, H. Yao, and C. Wang, Prime factorization algorithm based on parameter optimization of Ising model, Scientific Reports **10**, 10.1038/s41598-020-62802-5 (2020).

[6] M. Aramon, G. Rosenberg, E. Valiante, T. Miyazawa, H. Tamura, and H. G. Katzgraber, Physics-inspired optimization for quadratic unconstrained problems using a digital annealer, Frontiers in Physics **7**, 48 (2019).

[7] M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, *et al.*, Quantum annealing with manufactured spins, Nature **473**, 194 (2011).

[8] H. Goto, K. Tatsumura, and A. R. Dixon, Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems, Science Advances **5**, eaav2372 (2019).

[9] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, Optimization by simulated annealing, Science **220**, 671 (1983).
[10] T. Preis, P. Virnau, W. Paul, and J. J. Schneider, GPU accelerated Monte Carlo simulation of the 2D and 3D Ising model, Journal of Computational Physics **228**, 4468 (2009).
[11] I. G. Rosenberg, Reduction of bivalent maximization to the quadratic case, Cahiers du Centre d'Études de Recherche Opérationnelle **17**, 71 (1975).
[12] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. (Society for Industrial and Applied Mathematics, 2003).
[13] D. J. A. Welsh and M. B. Powell, An upper bound for the chromatic number of a graph and its application to timetabling problems, The Computer Journal **10**, 85 (1967).
[14] W. Ben-Ameur, Computing the initial temperature of simulated annealing, Computational Optimization and Applications **29**, 369 (2004).

Appendix A: Reduction rules

Every rule below acts on a single column clause, which is a linear (or low-degree) equation in binary variables $x_i \in \{0,1\}$. Each either fixes a variable to a constant or links it to an expression in the others. Substituting the result back into the remaining clauses shrinks the problem. We group the rules by which of these two things they do.

1. Rules that fix a variable

a. Saturated sum. If $\sum_{i=1}^{n} x_i = n$ (all n coefficients $+1$), then every $x_i = 1$ (the n binary variables sum to n only when all are one). The sign-flipped form $\sum_i(-x_i) = -n$ gives the same conclusion.

b. Zero sum. If $\sum_{i=1}^{n} x_i = 0$ with same-sign coefficients, then every $x_i = 0$ (a sum of non-negative terms vanishes only when each term equals to zero).

c. Coefficient bound. For a clause $\sum_i c_i x_i + C = 0$, let $S_+ = \sum_{c_i>0} c_i$ and $S_- = \sum_{c_j<0} |c_j|$. A single coefficient that exceeds the largest sum the opposing terms can form, forces its variable:

$$c_i > S_- - C \Rightarrow x_i = 0, \qquad c_i > S_+ + C \Rightarrow x_i = 1$$

for $c_i > 0$ (the negative-coefficient cases are symmetric in S_+ and S_-). In either case no assignment of the remaining variables can balance the clause otherwise.

2. Rules that link variables

a. Doubling. $x_1 + x_2 = 2x_3$ forces $x_1 = x_2 = x_3$. The left side lies in $\{0,1,2\}$ and the right in $\{0,2\}$, so the left side must be even; two binaries sum to an even number only when they are equal, and that shared value is exactly x_3.

b. Complement. $x_1 + x_2 = 1$ has only the solutions $(0,1)$ and $(1,0)$, so $x_2 = 1 - x_1$ and the product $x_1x_2 = 0$. This eliminates one variable outright and kills any quadratic term x_1x_2 in clauses that referenced both.

c. Parity. Both sides of a clause must agree modulo 2. In a column clause every carry term carries an even coefficient, so the parity of the small factor-bit sum is pinned by the parity of N_i: two factor bits with odd coefficients are forced equal when N_i is even, and complementary ($x_1 + x_2 = 1$) when N_i is odd.

3. The replacement rule

a. Replacement. When no fixing or linking rule fires, a final *replacement* rule keeps the simplification moving: it isolates a carry variable s that appears with coefficient ±1 in some clause, solves that clause for s, and substitutes the resulting expression into every other clause. Unlike the rules above it removes a variable unconditionally rather than only under a numeric condition, which makes it powerful but greedy, a long chain of cascading replacements can fuse many small clauses into a single dense clause whose square inflates the QUBO. We guard against this with a checkpoint: the simplifier saves its state before the first replacement and restores it if the chain fails to produce any new value assignments, so replacement is kept only when it actually shrinks the problem.

4. The power rule

a. Idempotence. For any $x \in \{0,1\}$ and integer $m \geq 1$, $x^m = x$. Squaring the energy in Eq. (3) therefore collapses every p_k^2, q_k^2, and s^2 back to its linear form, which is what lets the squared objective be written as a degree-2 QUBO once the residual cubic and quartic terms are quadrized (Section III B).

Appendix B: Implementation details

This appendix expands the two implementation choices summarized in Section V C, and records the start-temperature estimate, cooling schedule, and correctness checks used in every run.

1. Dense and sparse spin bins

The two empirical features from Section IV — a small dense hub set and a long sparse tail — map naturally onto two GPU kernels.

Dense bin: (row degree $\geq$ 128). One CUDA block of 1024 threads handles one spin. Each thread accumulates a partial sum over a stride of the spin's neighbor row; a shared-memory tree reduction collapses the partials to ΔE. The block size matches the dense-row length so threads stay busy.

Sparse bin: (row degree < 128). One warp (32 threads) handles one spin; 32 spins are packed into a 1024-thread block. Each warp does a warp-shuffle reduction — no shared memory, no explicit synchronization within the warp. This gives much higher occupancy than the dense layout would for short rows.

The threshold 128 matches the warp's natural reduction tier and is robust across the bit-width range we tested: above it, dense reduction amortizes the block-launch overhead; below it, the warp layout wins on occupancy.

2. Hub-tagged sparse CSR

The dense (hub) spins are read by essentially every sparse spin in every color pass when calculating their ΔE. Reading them from global memory each time is expensive, so we cache them once per color pass.

Hub values for the entire problem are copied into a shared-memory array (at most 256 bytes total) at the start of each sparse kernel launch. To resolve "is this neighbor a hub or not?" branchlessly, we tag the column-index array of the sparse-only CSR: the high bit of `col_idx[k]` encodes the answer.

- High bit set: the low 31 bits are an index into the hub cache in shared memory.
- High bit clear: the entry is a raw global spin id, used as a fallback for the rare non-hub neighbor.

The inner neighbor-sum loop is therefore a sign test on `col_idx[k]`, a single shared-memory load on the hot path, and a global-memory load on the cold path. Whenever a hub spin flips (which can only happen on a color pass that contains hubs), the hub-value cache is refreshed by a small dedicated kernel before any sparse pass reads it.

3. Auto start temperature and cooling

A simulated-annealing run is parameterized by a temperature schedule $T_t = T_0\alpha^t$ with $0 < \alpha < 1$ and an inner sweep count. The wrong T_0 wastes sweeps: too high and the run is effectively a random walk for many iterations; too low and the system freezes into the initial basin.

We estimate T_0 adaptively, as done in [14]. Across $K = 10$ random spin configurations we sample every spin's uphill ΔE (the energy cost of a single flip when it raises the energy), average them, and set

$$T_0 \;=\; \langle \Delta E^+ \rangle \,/\, \ln(1/\chi), \qquad \text{(B1)}$$

where $\chi = 0.5$ is the target acceptance rate at the start of the schedule. The sampling reuses the same GPU buffers and hub cache as the annealing loop, so the cost is at most one full sweep per configuration. Cooling is geometric with $\alpha = 0.95$; the schedule runs until $T < 10^{-3}$.

4. Numerical correctness

Symmetry check.: On load, the annealer samples random rows of the CSR and verifies that for every stored (i, j, J_{ij}) the partner (j, i, J_{ji}) is also stored and has the same value. The energy and ΔE formulas assume this; an upper-triangular-only CSR would silently halve every off-diagonal coupling.

Running-energy accumulator.: Instead of recomputing the total energy from scratch after each sweep, the annealer keeps a running sum: every accepted flip atomically adds its ΔE to a `double` accumulator. At the end of each annealing run, the accumulator is checked against a fresh full-state recomputation; relative drift stays below 10^{-9} for all problem sizes we tested.